

Reviewer Pack — Minimal Symmetric Tensor Rank vs SAT Proof Length

Entry 268150db97d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 13:26:58 UTC

Minimal Symmetric Tensor Rank vs SAT Proof Length

Entry ID: 268150db97d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 13:26:58 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For every CNF formula, the minimal symmetric tensor rank of its corresponding Gram matrix is upper bounded by a polynomial in the number of variables and clauses.’, ‘sentence_2’: ‘The length of any resolution proof for the CNF formula is lower bounded by an exponential function of the minimal symmetric tensor rank of its Gram matrix.’, ‘sentence_3’: ‘For all instances with $n \leq 40$, this relationship holds with a high statistical confidence using 30 random seeds.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Symmetric function theory provides a rich framework for studying the algebraic structure of polynomials, which may reveal hidden structures in Boolean functions.’, ‘sentence_2’: ‘The connection to resolution proof length suggests that symmetric tensors might offer insights into the complexity of solving SAT problems.’, ‘sentence_3’: ‘This conjecture aims to bridge these two fields and potentially discover new ways to approach complexity lower bounds.’}

Taxonomy category: TENSOR_RANK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fc96e4ee605af717

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each instance, if the ratio of the average minimal symmetric tensor rank to the average SAT proof length is less than or equal to a polynomially bounded threshold (e.g., $\leq 10^{(1/2)}$) across all 30 seeds and there is no seed where this ratio exceeds that threshold, the conjecture is supported. Falsification occurs if any seed has a ratio greater than the threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal symmetric tensor rank" AND "resolution proof complexity"
- "Gram matrix" AND "symmetric function theory" AND "proof length" - "CNF formula" AND
"symmetric tensor rank" AND "exponential lower bound"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for _ in range(m):
            clause = random.sample(variables + [-v for v in variables], 3)
            clauses.append(clause)
        return clauses

    def gram_matrix(cnf):
        n = len(cnf)
        M = [[0] * n for _ in range(n)]
        for i, clause in enumerate(cnf):
            for j, clause2 in enumerate(cnf):
                count = sum(1 for v in clause if -v in clause2)
                M[i][j] = count
        return M

    def smith_normal_form(M):
        n = len(M)
        U = [[0] * n for _ in range(n)]
        V = [[0] * n for _ in range(n)]
        for i in range(n):
            U[i][i] = 1
```

```

V[i][i] = 1

def pivot(M, r, c):
    M[r], M[c] = M[c], M[r]
    for j in range(c+1, n):
        M[r][j] /= M[r][c]
    for i in range(n):
        if i != r:
            factor = M[i][c]
            for j in range(c, n):
                M[i][j] -= factor * M[r][j]

def find_pivot(M):
    for i in range(n):
        for j in range(n):
            if M[i][j] != 0:
                return i, j
    return None

while True:
    pivot_row, pivot_col = find_pivot(M)
    if pivot_row is None:
        break
    pivot(M, pivot_row, pivot_col)

for r in range(n):
    for c in range(r+1, n):
        M[r][c] = 0

return M

def resolution_length(cnf):
    stack = cnf[:]
    while True:
        new_clause = None
        for i in range(len(stack)):
            for j in range(i+1, len(stack)):
                clause_i = set(stack[i])
                clause_j = set(stack[j])
                if any(-v in clause_i for v in clause_j):
                    new_clause = [v for v in clause_i | clause_j if v not in [-v for v in clause_i]]
                    break
            if new_clause:
                break
        if not new_clause:
            return len(stack)
        stack.append(new_clause)

n = random.randint(5, 40)
m = random.randint(n, 2 * n)
cnf = generate_cnf(n, m)
gram = gram_matrix(cnf)
snf = smith_normal_form(gram)
rank = sum(1 for row in snf if any(x != 0 for x in row))

proof_length = resolution_length(cnf)

```

```

    return {
        "metric_name": "Ratio of Rank to Proof Length",
        "metric_value": rank / proof_length,
        "instances_tested": 1,
        "conjecture_holds": rank <= 10 ** (1/2) * proof_length,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [random.randint(1, 100000) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank > sqrt(proof_length)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support or budget exceeded")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eb14c73b.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eb14c73b.py", line 94, in run_trial
    cnf = generate_cnf(n, m)
         ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eb14c73b.py", line 25, in generate_cnf
    clause = random.sample(variables + [-v for v in variables], 3)
                        ^

```

TypeError: bad operand type for unary -: 'str'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the pre-registered support conditions. | next: Investigate and fix the error in the test code to allow for proper execution and verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14402
2	preregistration	ollama_remote	glm4:latest	0	0	9314
3	novelty	ollama_remote	glm4:latest	0	0	8339
4	novelty	ollama_remote	glm4:latest	0	0	9134
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20037
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11603
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11481
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12278
9	judge	ollama_remote	glm4:latest	0	0	13884

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110473 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/268150db97d3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/268150db97d3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/268150db97d3.tar.gz (if generated)